Pyrmit: architecture, needs and workflows

October 25, 2026. How the project runs today, what it needs, where money is needed, and how a team of people and coding agents can share the same corpus, databases and budget. Current stack: GitHub, Linear, Neon, plus OpenRouter for the LLM and one development PC with a GPU. Sizes and timings were measured on that PC on October 25; prices come from the hosting report of October 9 (third-party trackers, to check before paying).

Summary

- **Today one PC does everything**: the offline pipeline (crawl, OCR, indexing) and the app. Neon holds users and chats, OpenRouter the LLM. GitHub has the code, Linear the issues. CI runs only the frontend lint.
- **The data grew past a laptop**: 1.9M chunks, a 7.4 GB corpus store and 194 GB of crawled PDFs, 5 to 10× the October 9 figures. It should live in object storage and travel as **versioned corpus snapshots**, built once on the GPU machine and pulled by everyone else.
- **Money goes to** LLM tokens first (about $10 per 1,000 answers, $2–3 per benchmark round), then object storage (about $3 a month for the PDFs), a server (CPU, price to check) and optional GPU rental for peaks. Neon, GitHub and Linear stay on their current plans.
- **Pool what is expensive or slow**: one corpus build shared as snapshots, one LLM account with a capped key per person, agent and environment, one GPU queue that runs at night, one Neon project with a branch per PR and worktree.
- **Two kinds of data, never mixed**: mutable app state (Neon, branched per environment) and immutable corpus snapshots (files, pinned per environment, promoted after checks).
- **Worktrees**: each gets its branch, its Neon branch and its ports, and all share one read-only data folder. Only one backend can hold the full store on the 8 GB GPU; the others run on the CPU or on the small corpus.
- **First steps cost nothing**: backend tests in CI, Neon branches per PR, capped OpenRouter keys. Then snapshots on R2 and a staging server.

1. The system today

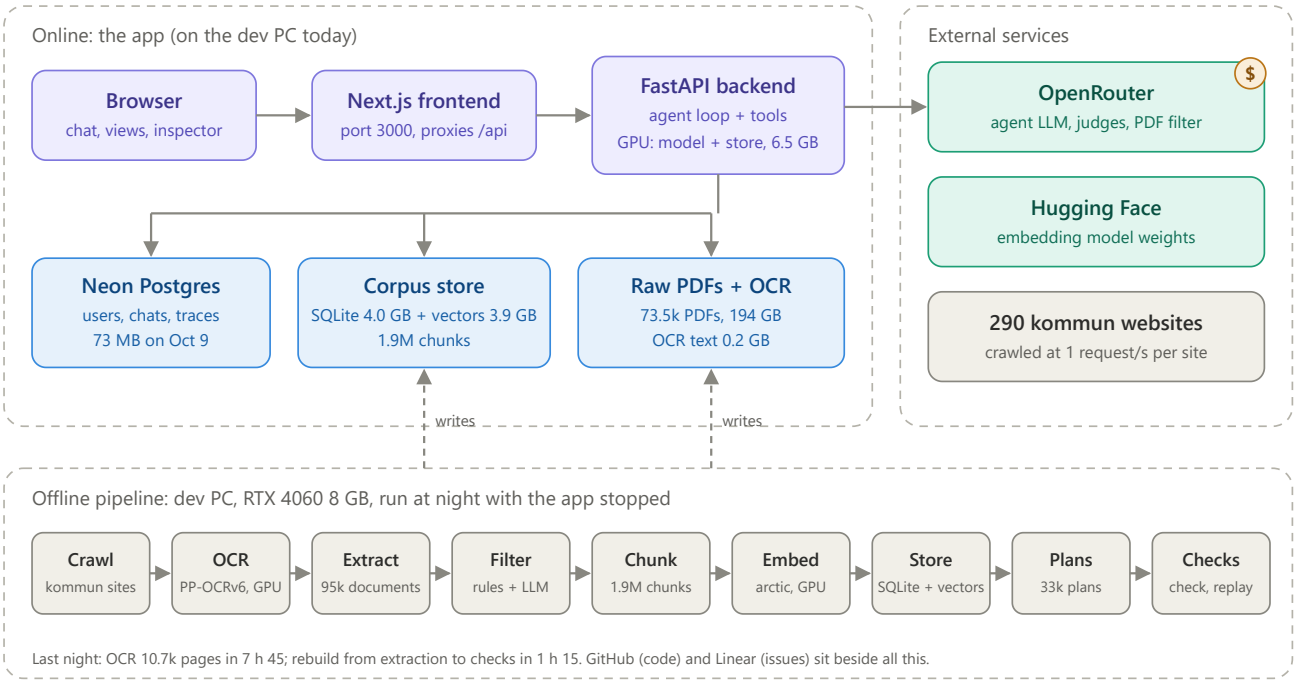


Figure 1. The system on October 25, 2026. ▢ money is spent here

Part	Size or need (measured Oct 25)	Where it lives
Corpus store (`data/index/store/st-arctic-l-v2/`)	corpus.sqlite 4.0 GB + vectors.npy 3.9 GB (1.9M × 1024 float16); read-only, built by the pipeline	dev PC disk; copied to the GPU at backend start
Raw data (`data/corpus/raw/web/`)	73.5k PDFs, 194 GB; 41k web pages 0.1 GB; OCR outputs 0.2 GB	dev PC disk
Rebuild inputs	documents.jsonl 2.2 GB, chunks.jsonl 2.7 GB, dense index 6.1 GB	dev PC disk
Small corpus (`data/small`)	the October 22 corpus, 3.6 GB (PDFs shared with the full one)	dev PC disk
App data	users, chats, traces (73 MB on Oct 9; 20–60 KB per answer)	Neon
Backend at runtime	about 6.5 GB of the 8 GB GPU; on a CPU the ONNX int8 query encoder (34 ms, 0.9 GB RAM)	dev PC
CI	frontend lint on pushes and PRs to main; no backend tests	GitHub Actions

2. Target organisation

The draft below keeps four layers. Planning decides what is done, development does it (people and agents, in parallel), shared services are pooled across everyone, and environments run the result. The dollar badges mark where money is spent.

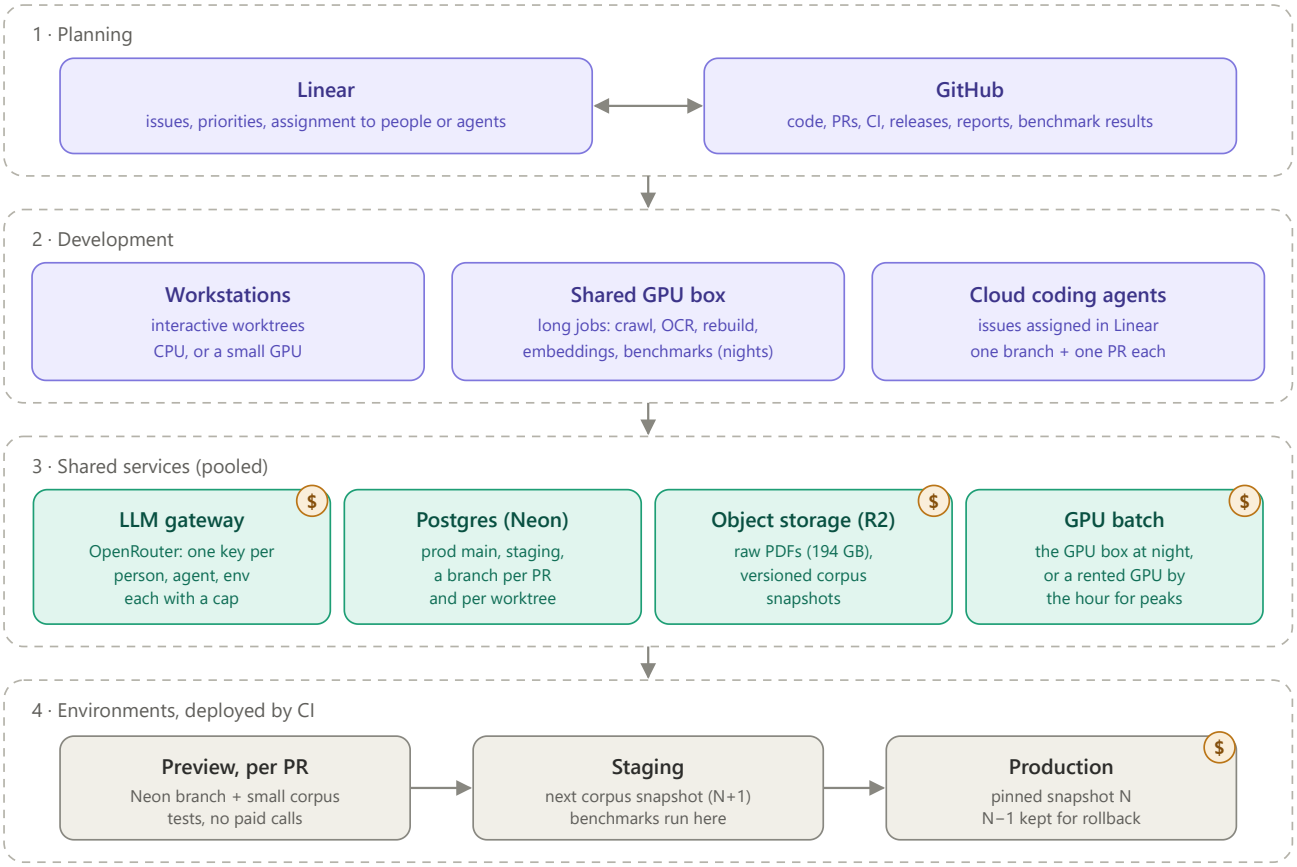


Figure 2. Target layers (from the team's draft). Production costs a server plus the LLM tokens of its users.

- **Planning**: Linear holds every task, including those given to agents. The Linear–GitHub integration links branches and PRs to issues and moves their status, so a PR is always traceable to a decision.
- **Development**: people work interactively in worktrees; long jobs (crawl, OCR, rebuild, re-embedding, paid benchmarks) never run on laptops but on the shared GPU box, at night; cloud agents take well-scoped issues and open PRs that a person reviews.
- **Shared services**: one account each, split by keys, branches and folders rather than duplicated per person.
- **Environments**: previews prove the code, staging proves the next corpus, production only changes through these two.

3. Needs, and where money is needed



Figure 3. Needs grouped by kind, coloured by how they are paid.

Item	Estimate	Basis
LLM, users' answers	~$10 per 1,000 answers	$0.007–0.011 per agent answer (GPT-6 Luna via OpenRouter, benchmark data)
LLM, evaluation	~$1.6–2.9 per round (dev set × 3 passes + second judge)	the optimisation and bug rounds of Oct 21–22
Object storage	~$2.8/month for 194 GB of PDFs; ~$0.11 per 7.4 GB snapshot	Cloudflare R2: $0.015 per GB-month after 10 GB free, no egress fee
Server	to price: CPU, 16 GB RAM, ~250 GB disk (or PDFs read from R2 and ~40 GB disk)	the Oct 9 estimate (€7–15/month) was for 8 GB RAM and a 1.5 GB store; the store is now 7.4 GB
GPU	own PC: the OCR backlog takes ~3–4 nights; rented: a few dollars per backlog	last night's 10.7k pages in 7 h 45; RTX 4090 ~$0.34/h quoted, not tried
Corpus in pgvector (option)	~8–11 GB, ~$3–4/month of storage on Neon Launch, plus compute	the Oct 9 footprint (1.5–2 GB for 348k chunks) scaled to 1.9M; estimate

4. Pooling the needs

The expensive or slow things are built once and shared; the cheap things are split per person by keys, branches and folders, never by copying the whole stack.

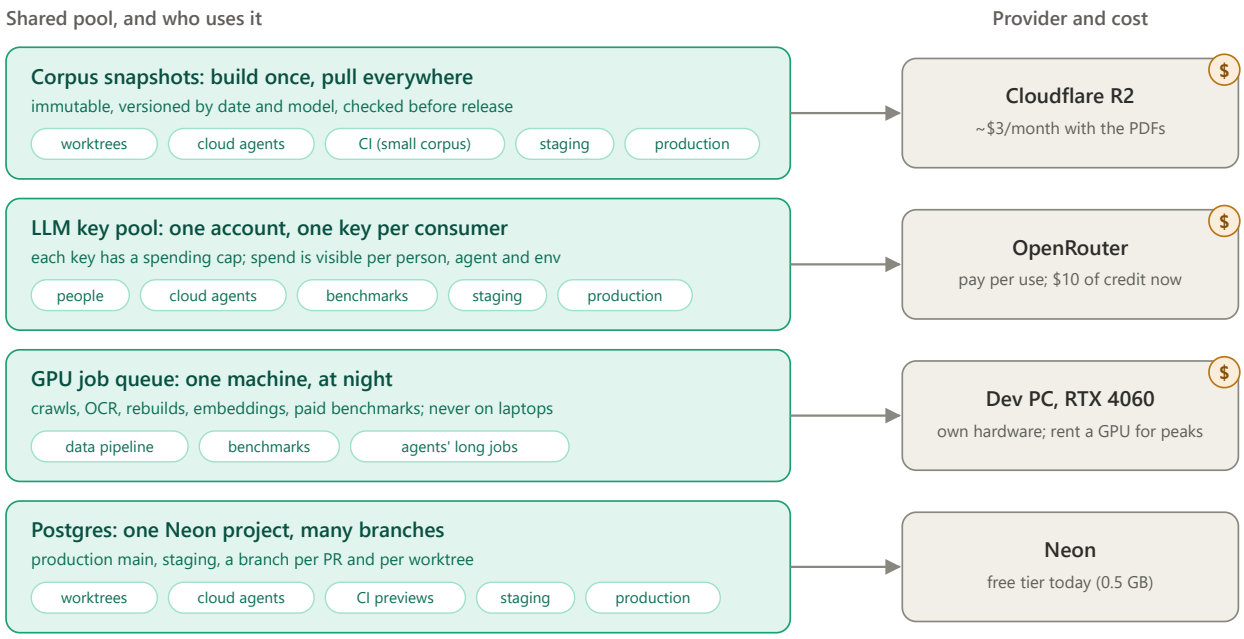


Figure 4. Four pools serve every consumer; the providers on the right are the only bills.

- **Corpus snapshots** replace "send the data folder": a rebuild costs GPU hours, a download costs minutes. Everyone (people, agents, CI, staging, prod) reads the same immutable files, so a bug report names a snapshot.
- **Capped keys**: a runaway agent or benchmark can only spend its own cap (Windows keeps killed Python jobs alive: a "stopped" benchmark kept spending once). The product's keys stay separate from people's experiments.
- **One GPU queue**: the 8 GB card holds either the app (6.5 GB) or a batch job, not both. Batch jobs are queued for the night, with the app stopped, as the night script already does.
- **Neon branches** are copies on write: a branch per PR or worktree costs almost nothing and isolates schema changes and test data.

5. Databases and data versions

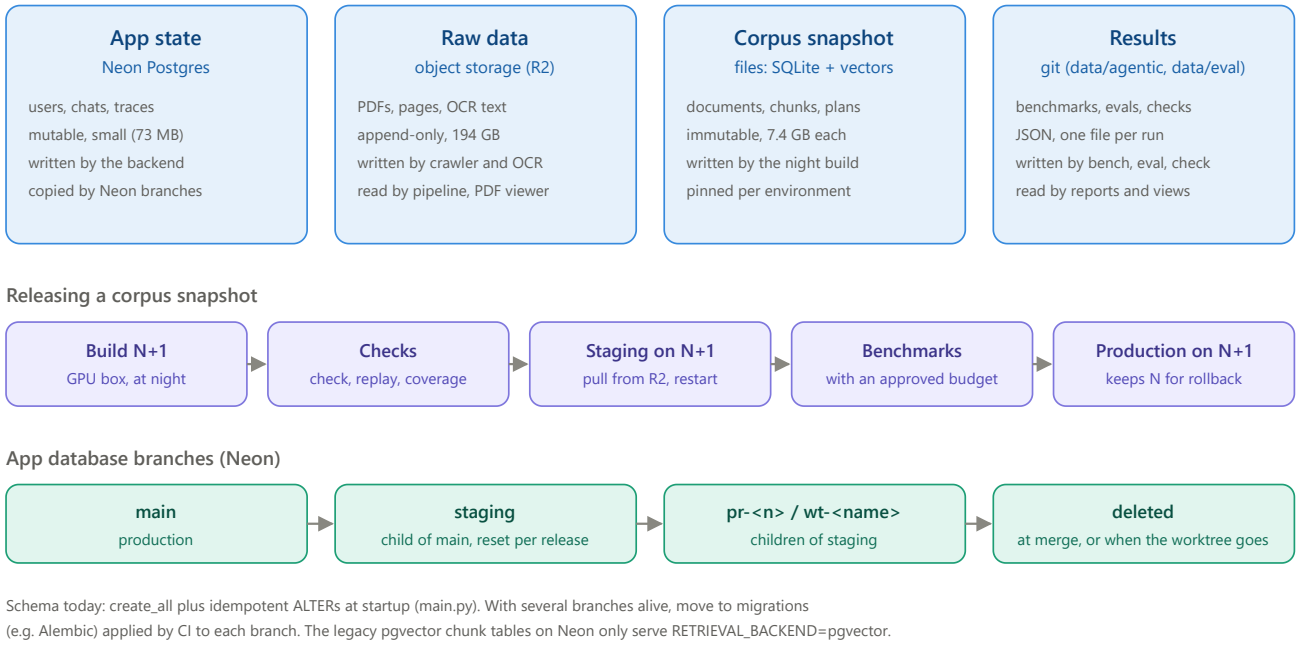


Figure 5. Four stores with one writer each; corpus snapshots and app branches move through the same gates.

- **Never mix app state and corpus**: the corpus is rebuilt from raw data at any time, the app state is not. Keeping the corpus out of Neon also keeps Neon in its free tier (1.9M chunks would take ~8–11 GB there).
- **A snapshot is a folder** such as `corpus-20261025-arctic-l-v2/` holding the store, the plan registry, `check.json` and a manifest of counts; `CORPUS_DATA_DIR` already selects one, as `data/small` does today.
- **Raw data is the source of truth** for rebuilds: append-only in a bucket, so any snapshot can be rebuilt, and the PDF viewer and map tools read from it through a local cache.
- **Results stay in git**: they are small, and the reports are generated from them.

6. Workflows

6.1 Code, for people and agents alike

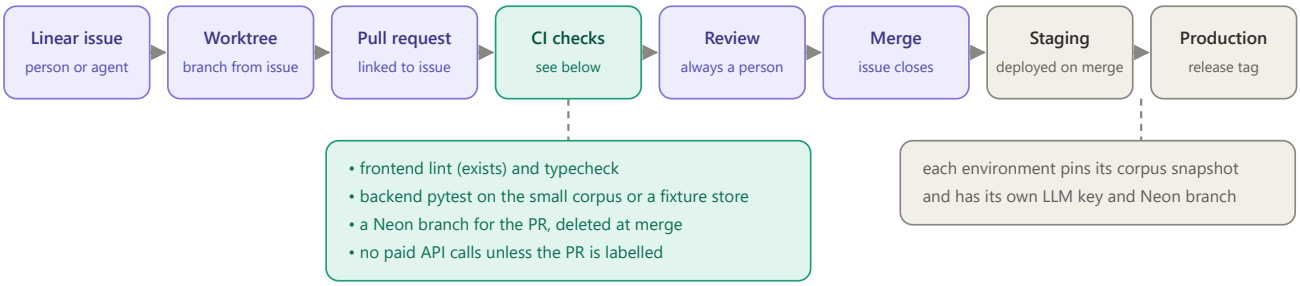


Figure 6. One path to production. Agents follow it like people; their PRs always get a human review.

6.2 Data, on the GPU box

- **Weekly**: `corpus.crawl --revisit` (new PDF links on known pages, ~35k requests, about an hour); a full crawl (~166k requests, several hours) monthly or when the crawl rules change.
- **Nightly, while the OCR backlog lasts**: OCR until a deadline, then the rebuild (extraction → filter → chunks → embeddings → store → plans → checks → replay), as the night script does; then publish snapshot N+1 to R2.
- **Release**: staging switches to N+1; the coverage benchmark and, with a budget, the agent benchmarks run there; production switches when they hold. A failed check stops the release, not the night.

6.3 Evaluation

- Every paid run has a label and runs on its own capped key; results (JSON) are committed and the reports are regenerated from them, so a number in a report always points to a file.
- Free checks run on every snapshot (data checks, tool replay, coverage); paid benchmarks run when the prompts, the model or the corpus change in a way the free checks cannot judge.

7. Worktrees and agents in parallel

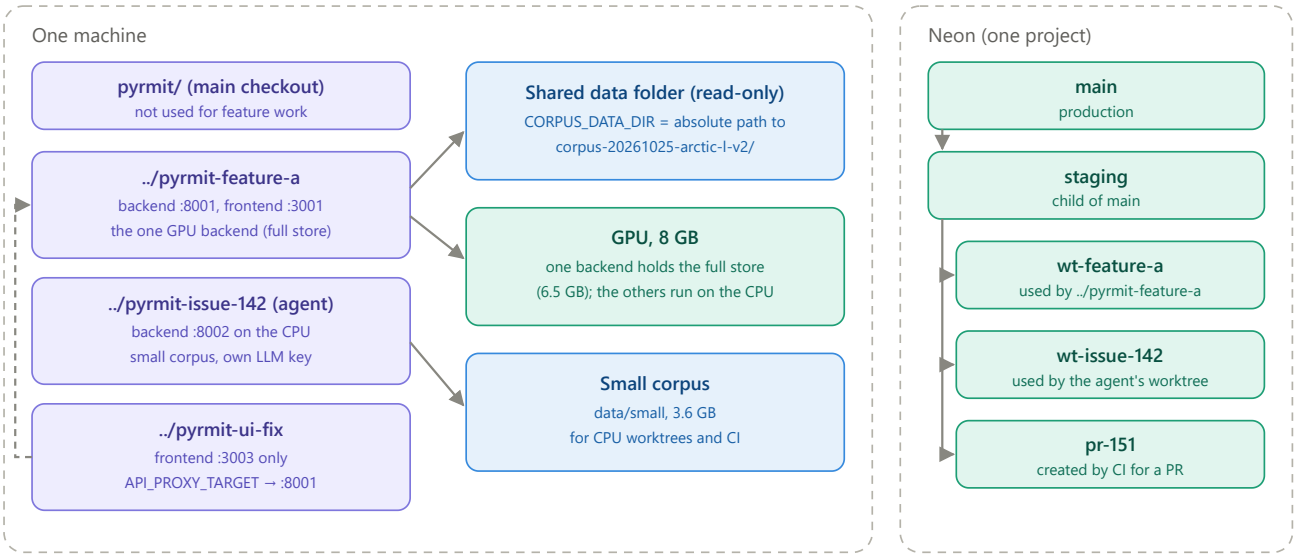


Figure 7. Three worktrees on one PC: one owns the GPU, one runs on the CPU and the small corpus, one only changes the frontend and borrows a backend. Each has its own Neon branch.

Setting	Production	Staging	PR preview (CI)	Worktree
`DATABASE_URL`	Neon main	branch `staging`	branch `pr-<n>`	branch `wt-<name>`
`CORPUS_DATA_DIR`	snapshot N	snapshot N+1	small corpus or fixture	shared folder, full or small
LLM key	production key	staging key	none, or a CI key with a low cap	the person's or agent's key
GPU	optional	optional	none	one backend at a time
Ports	–	–	–	8000+n and 3000+n

```
# bash syntax
git worktree add ../pyrmit-feature-a -b feature-a            # branch named from the Linear issue
neonctl branches create --name wt-feature-a --parent staging  # or the Neon console / API
# ../pyrmit-feature-a/backend/.env : DATABASE_URL=<branch URL>, CORPUS_DATA_DIR=<shared snapshot>, own
OPENROUTER key
# CPU backend when another worktree holds the GPU (CUDA_VISIBLE_DEVICES=-1: on Windows an empty value
deletes it)
CUDA_VISIBLE_DEVICES=-1 EMBEDDING_RUNTIME=onnx python -m uvicorn main:app --port 8002
cd frontend && API_PROXY_TARGET=http://localhost:8002 bunx next dev -p 3002
```

- **One GPU backend per 8 GB card**: the store and model take 6.5 GB. Measured with the GPU hidden on Oct 25: a search in one kommun takes 0.03–0.14 s on the CPU and in the statutes 0.03 s, but a search over the whole country 13–16 s (3.9 GB of vectors read per query). CPU worktrees should use the small corpus or questions that name a kommun.
- **Ports and stale processes**: on Windows a stopped shell leaves its Python and Node children running; each worktree uses its own ports, and processes are checked and stopped before a restart.
- **Agents**: one issue, one worktree, one branch, one Neon branch, one capped key. Long jobs (OCR, full crawls, re-embedding, paid benchmarks) are not run by agents: they are queued on the GPU box.
- **Shared data is read-only**: only the night build writes a new snapshot folder, so worktrees never see a half-written store.

8. Next steps

#	Step	Cost	Unlocks
1	Backend tests and frontend typecheck in CI, on the small corpus or a fixture store	free	agent PRs that are safe to review
2	Neon: a `staging` branch, a branch per PR (GitHub Action) and per worktree	free tier, check limits	parallel work without shared test data
3	OpenRouter: one key per person, agent and environment, each with a cap	free	visible, bounded LLM spend
4	Corpus snapshots and raw PDFs on R2, with a pull script	~$3/month	no more data folders sent by hand
5	A staging server pinned to a snapshot; an ANN index (or a GPU) for nationwide searches on the CPU	server: to price	a shareable app off the dev PC
6	GPU queue: finish the OCR backlog at night (~3–4 nights), rent a GPU for peaks	own PC, a few $ rented	every scanned plan searchable
7	Pilot the map-application crawl agent on 2–3 map-only kommuner	to measure	the plans a third of the kommuner publish only in a map
8	Schema migrations once several Neon branches live	work	safe schema changes across branches

Caveats and sources

- Sizes and timings: measured on the development PC (RTX 4060 Laptop 8 GB, 16 GB RAM) on October 25, 2026, and from last night's run log (OCR, rebuild).
- Prices: `reports/pyrmit_hosting_report.md` (October 9, 2026: Cloudflare R2 official; Neon, VPS and other prices from third-party trackers, to check before paying). The rented-GPU price is a quote, not tried. LLM costs come from the project's benchmark result files (`backend/data/agentic/`).
- Coverage figures (map-only kommuner, missing plans): benchmark report section 2.11.
- Tool features named here (Linear–GitHub integration, Neon branching and its CLI, OpenRouter per-key limits) should be checked against each plan before relying on them.